Bloque 2: conteo y recursión

Programación: concepto, ejemplo y problema de juez

Carlos A Delgado S

Pontificia Universidad Javeriana Cali
Sesiones de refuerzo de algoritmia

Los temas

Cómo va cada tema

Tres partes en el mismo orden: el concepto con lo que cuesta, un ejemplo pequeño con su traza en una tabla, y el problema de UVa donde se usa, resuelto entre todos y escrito en Python y en C++.

Tema	Problema
Contar operaciones	UVa 12149, *Feynman*
Recursión y árbol de llamadas	UVa 839, *Not so Mobile*
Recursión por índices y su costo	UVa 11173, *Grey Codes*

Compilar, correr y comparar

C++

```
g++ -std=c++17 -Wall -Wextra solucion.cpp -o solucion
./solucion < 1.in > salida.out
diff salida.out 1.out
```

Python

```
python3 solucion.py < 1.in > salida.out
diff salida.out 1.out
```

Contenido

1 Contar operaciones

2 Recursión y árbol de llamadas

3 Recursión por índices y su costo

4 Tarea del bloque

Contenido

1 Contar operaciones
- Concepto
- Ejemplo
- Ejercicio: UVa 12149

2 Recursión y árbol de llamadas

3 Recursión por índices y su costo

4 Tarea del bloque

Instrucciones, no segundos

Pregunta

Un programa tarda 2 segundos en un computador. ¿Cuánto tarda en otro?

Respuesta

No se sabe: el tiempo depende de la máquina. Lo que no cambia de un computador a otro es cuántas instrucciones ejecuta el programa. Por eso se cuenta, línea por línea, cuántas veces corre cada una, se suma y el total se escribe como una función $T(n)$ del tamaño de la entrada.

El costo de cada línea

Una asignación, una operación aritmética, una comparación o un acceso a memoria cuestan lo mismo: 1. Una declaración como `int i;` también cuenta 1.

Las reglas del conteo

La condición de un ciclo

Se cuenta cada vez que se evalúa, también la última, la que da falso y saca del ciclo. Un ciclo de n vueltas evalúa su condición $n + 1$ veces.

La cabecera del `for`

`for (i = 0; i < n; ++i)` cuenta como su condición: $n + 1$. La inicialización y el incremento no se cuentan aparte. Con `while` sí se cuentan: la condición $n + 1$, el cuerpo n y el incremento n. En Python, `for i in range(n)` cuenta como el `for` de C++.

Cuando una línea depende de los datos

Se le pone una variable, como k, y se buscan los datos que dan el menor y el mayor $T(n)$: el mejor y el peor caso.

Cada $T(n)$ se comprueba

Se escoge un n pequeño, se cuenta a mano corriendo el código y se compara con la fórmula. Si no coinciden, una de las dos está mal.

Ciclos anidados y las sumas que hacen falta

Congelar el índice de afuera

Con dos ciclos anidados, se fija un valor de i y se cuenta el de adentro como función de i. Después se suma sobre todos los i.

Las tres sumas cerradas

$$\sum_{i=0}^{n-1} 1 = n \qquad \sum_{i=1}^{n} i = \frac{n(n+1)}{2} \qquad \sum_{m=1}^{N} m^2 = \frac{N(N+1)(2N+1)}{6}$$

Y la linealidad: $\sum(a_i + b_i) = \sum a_i + \sum b_i$ y $\sum c \cdot a_i = c \sum a_i$.

Anidar multiplica solo si el de adentro no depende del de afuera

Con `for (j = i; j < n; ++j)` el ciclo de adentro da menos vueltas en cada i, y el total no es $n \cdot n$.

Sumar un arreglo

C++

```cpp
int sumarElementos(vector<int> &A) {
    int ans = 0;
    int i;
    for (i = 0; i < (int) A.size(); ++i) {
        ans += A[i];
    }
    return ans;
}
```

Línea	Veces
`int ans = 0;`	1
`int i;`	1
`for (i = 0; i < n; ++i)`	?
`ans += A[i];`	?
`return ans;`	1

Sumar un arreglo

Línea	Veces
`int ans = 0;`	1
`int i;`	1
`for (i = 0; i < n; ++i)`	$n+1$
`ans += A[i];`	n
`return ans;`	1
$T(n)$	$2n+4$

Comprobación con $n = 2$

i toma los valores 0, 1 y 2: la condición se evalúa 3 veces y la suma corre 2. Total $1+1+3+2+1 = 8$, y la fórmula da $2 \cdot 2 + 4 = 8$.

Buscar un valor sin detenerse

C++

```cpp
bool buscarElemento(vector<int> &A, int v) {
    bool ans = false;
    int i;
    for (i = 0; i < (int) A.size(); ++i) {
        if (A[i] == v)
            ans = true;
    }
    return ans;
}
```

Pregunta

¿Cuántas veces corre `ans = true`?

Buscar un valor sin detenerse

Respuesta

Depende de los datos: tantas veces como v aparezca en A. Se llama k, con $0 \leq k \leq n$.

Línea	Veces
`bool ans = false;` y `int i;`	2
`for`	$n+1$
`if (A[i] == v)`	n
`ans = true;`	k
`return ans;`	1
$T(n)$	$2n+k+4$

Mejor y peor caso

Mejor: v no está, $k=0$ y $T(n)=2n+4$. Peor: todo el arreglo vale v, $k=n$ y $T(n)=3n+4$. Los dos crecen como n.

Buscar un valor y detenerse

C++

```
bool buscarElemento2(vector<int> &A, int v) {
    bool ans = false;
    int i;
    for (i = 0; i < (int) A.size() && !ans; ++i) {
        if (A[i] == v)
            ans = true;
    }
    return ans;
}
```

Pregunta

Con $A = [3, 4, 5, 1, 6]$ y $v = 4$, ¿cuántas veces se evalúa la condición del `for`?

Buscar un valor y detenerse

Respuesta

Tres: con $i = 0$ y con $i = 1$ da verdadero; con $i = 2$, `ans` ya vale `true` y da falso. Ni la condición ni `ans = true` se pueden contar sin los datos: se llaman y y z.

Línea	Veces
declaraciones	2
`for`	y
`if`	$y - 1$
`ans = true;`	z
`return`	1
T	$2y + z + 2$

Buscar un valor y detenerse

Mejor caso: v en la posición 0

$y = 2$ y $z = 1$: $T = 7$, sin importar n.

Peor caso: v en la última posición o ausente

$y = n + 1$. Si está al final, $z = 1$ y $T(n) = 2n + 5$; si no está, $z = 0$ y $T(n) = 2n + 4$.

Con la muestra

$y = 3$ y $z = 1$: $T = 2 \cdot 3 + 1 + 2 = 9$. Contado línea por línea: $2 + 3 + 2 + 1 + 1 = 9$.

Un índice que se duplica

Python

```python
def algoritmo1(n):
    i = 1
    ans = 0
    while i <= n:
        ans += i
        i *= 2
    return ans
```

Pregunta

Con $n = 20$, ¿qué valores toma i? ¿Cuántas veces se evalúa `i <= n`?

Un índice que se duplica

Respuesta

$i = 1, 2, 4, 8, 16, 32$: seis evaluaciones, y la última da falso. Son las potencias $2^0, \ldots, 2^{\lfloor \log_2 n \rfloor}$ más una: $\lfloor \log_2 n \rfloor + 2$.

Línea	Veces
`i = 1 y ans = 0`	2
`while i <= n`	$\lfloor \log_2 n \rfloor + 2$
`ans += i`	$\lfloor \log_2 n \rfloor + 1$
`i *= 2`	$\lfloor \log_2 n \rfloor + 1$
`return`	1
$T(n)$	$3\lfloor \log_2 n \rfloor + 7$

Comprobación con $n = 20$

$\lfloor \log_2 20 \rfloor = 4$: la fórmula da 19, y a mano son $2 + 6 + 5 + 5 + 1 = 19$.

Sumar una matriz

C++

```cpp
int sumarMatriz(vector<vector<int>> &M, int n) {
    int i = 0, j, ans = 0;
    for (i = 0; i < n; ++i)
        for (j = 0; j < n; ++j) {
            ans += M[i][j];
            printf("%d ", M[i][j]);
        }
    return ans;
}
```

Pregunta

Congele i. ¿Cuántas veces se evalúa `j < n` para ese i? ¿Y en total?

Sumar una matriz

Respuesta

Para cada i, el `for` de adentro es un ciclo de n vueltas: $n+1$ evaluaciones. El de afuera da n vueltas, así que en total son $n(n+1)$. El cuerpo corre $n \cdot n$ veces.

Línea	Veces
`int i = 0, j, ans = 0;`	3
`for` de i	$n+1$
`for` de j	$n(n+1)$
`ans += M[i][j];`	n^2
`printf`	n^2
`return`	1
$T(n)$	$3n^2+2n+5$

Comprobación con $n = 2$

$3+3+6+4+4+1 = 21$, y $3 \cdot 4 + 4 + 5 = 21$.

Solo el triángulo de arriba

C++: el ciclo de adentro arranca en i

```
int sumarMatriz2(vector<vector<int>> &M, int n) {
    int i = 0, j, ans = 0;
    for (i = 0; i < n; ++i)
        for (j = i; j < n; ++j) {
            ans += M[i][j];
            printf("%d ", M[i][j]);
        }
    return ans;
}
```

i	valores de j	evaluaciones de `j < n`	vueltas
0	$0, \ldots, n$	$n+1$	n
1	$1, \ldots, n$	?	?
i	$i, \ldots, n$	?	?
$n-1$	$n-1, n$	?	?

Solo el triángulo de arriba

i	valores de j	evaluaciones de `j < n`	vueltas
0	$0, \ldots, n$	$n+1$	n
1	$1, \ldots, n$	n	$n-1$
i	$i, \ldots, n$	$n-i+1$	$n-i$
$n-1$	$n-1, n$	2	1

Sumar sobre i

$$\sum_{i=0}^{n-1}(n-i) = n + (n-1) + \cdots + 1 = \frac{n(n+1)}{2}$$

$$\sum_{i=0}^{n-1}(n-i+1) = \frac{n(n+1)}{2} + n$$

Solo el triángulo de arriba

Línea	Veces
declaraciones	3
`for` de i	$n+1$
`for` de j	$n(n+1)/2+n$
las dos líneas del cuerpo	$2 \cdot n(n+1)/2$
`return`	1
$T(n)$	$(3n^2+7n)/2+5$

Comprobación con $n = 2$

$3+3+(3+2)+2 \cdot 3+1 = 18$, y $(12+14)/2+5 = 18$.

Multiplicar habría dado otra cosa

Con "n vueltas por n vueltas" el cuerpo daría $2n^2$; con $n = 2$, 8 en lugar de 6.

Tres bloques en secuencia

C++

```
int tresBloques(vector<int> &A, int n) {
    int i, j, ans = 0;
    for (i = 0; i < n; ++i)
        ans += A[i];
    for (i = 0; i < n; ++i)
        for (j = i; j < n; ++j)
            ans += A[j];
    i = 1;
    while (i <= n) {
        ans += i;
        i *= 2;
    }
    return ans;
}
```

Pregunta

Cada bloque ya está contado en los ejemplos anteriores. ¿Cuánto vale $T(n)$ de la función completa?

Tres bloques en secuencia

Respuesta: los bloques seguidos se suman

Parte	Veces
declaración	1
primer `for` y su cuerpo	$2n+1$
triángulo: `for` de i	$n+1$
triángulo: `for` de j	$n(n+1)/2+n$
triángulo: cuerpo	$n(n+1)/2$
`i = 1` y el ciclo que duplica	$1+3\lfloor \log_2 n \rfloor + 4$
`return`	1
$T(n)$	$n^2+5n+3\lfloor \log_2 n \rfloor + 9$

Comprobación con $n = 4$

$16 + 20 + 6 + 9 = 51$. El término que manda es n^2, el del bloque más caro.

UVa 12149: Feynman

Enunciado, en corto

¿Cuántos cuadrados distintos hay en una cuadrícula de $N \times N$? Se cuentan todos los tamaños: los de lado 1, los de lado 2 y así hasta el de lado N.

Entrada y salida

Un N por línea, $1 \leq N \leq 100$, hasta una línea con 0. Por cada N, una línea con la cantidad de cuadrados.

Direcciones

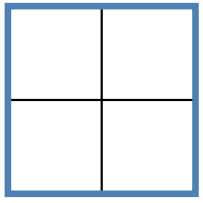
Enunciado: `https://onlinejudge.org/external/121/12149.pdf`

Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=3301`

¿Entendimos el problema?

Muestra

```
2
1
8
0
```



$N = 2$

Salida

```
5
1
204
```

A mano, $N = 2$

Cuatro cuadrados de lado 1 y uno de lado 2, el borde: 5.

Pregunta

Con $N = 8$, ¿cuántos cuadrados de lado 7 hay?

¿Entendimos el problema?

Respuesta

Cuatro. La esquina de arriba a la izquierda de un cuadrado de lado 7 puede estar en la fila 0 o en la 1, y en la columna 0 o en la 1: $2 \cdot 2 = 4$. En general, un cuadrado de lado ℓ tiene $N - \ell + 1$ filas posibles para su esquina y otras tantas columnas.

El último número

El 0 cierra la entrada y no se procesa: la muestra trae cuatro números y tres respuestas.

Primera idea: contarlos uno por uno

Python: `feynman_ciclos.py`

```python
def cuadrados(n):
    cuenta = 0
    for lado in range(1, n + 1):
        for fila in range(n - lado + 1):
            for col in range(n - lado + 1):
                cuenta = cuenta + 1
    return cuenta
```

Pregunta

¿Cuántas veces corre `cuenta = cuenta + 1`? Congele `lado`.

Congelar el lado

Traza con $N = 3$

Para un lado fijo, la fila y la columna van de 0 a $N - \ell$.

ℓ	vueltas de `fila`	vueltas de `col` por fila	cuerpo
1	3	3	9
2	?	?	?
3	?	?	?
total			?

Congelar el lado

ℓ	vueltas de `fila`	vueltas de `col` por fila	cuerpo
1	3	3	9
2	2	2	4
3	1	1	1
total			14

El conteo es la respuesta

Con el lado congelado, el cuerpo corre $(N - \ell + 1)^2$ veces. Al sumar sobre ℓ, y llamando $m = N - \ell + 1$:

$$\sum_{\ell=1}^{N} (N - \ell + 1)^2 = \sum_{m=1}^{N} m^2 = \frac{N(N+1)(2N+1)}{6}$$

Con $N = 8$: $8 \cdot 9 \cdot 17/6 = 204$, como en la muestra.

El conteo completo del triple ciclo

Pregunta

Con la cabecera de cada `for` contada como condición, ¿cuánto vale $T(N)$ de `cuadrados`?

Línea	Veces
`cuenta = 0`	1
`for lado`	$N+1$
`for fila`	$\sum_{m=1}^{N}(m+1)$
`for col`	$\sum_{m=1}^{N} m(m+1)$
`cuenta = cuenta + 1`	$\sum_{m=1}^{N} m^2$
`return`	1

El conteo completo del triple ciclo

Respuesta

Con las tres sumas cerradas:

$$T(N) = \frac{N(N+1)(2N+1)}{3} + N^2 + 3N + 3$$

Con $N = 2$ son $10 + 4 + 6 + 3 = 23$; a mano, $1 + 3 + 5 + 9 + 5 + 1 = 23$.

¿Alcanza?

El triple ciclo crece como N^3, y con $N = 100$ su cuerpo corre $338\,350$ veces por caso: alcanza. La fórmula responde con una cuenta, sin importar N, y es la que se envía.

Python: `feynman.py`

La fórmula

```python
def cuadrados(n):
    # suma de m * m para m de 1 a n: los cuadrados de lado n - m + 1
    return n * (n + 1) * (2 * n + 1) // 6
```

Leer hasta el 0

```python
def main():
    datos = sys.stdin.read().split()
    pos = 0
    n = int(datos[pos])
    while n != 0:
        print(cuadrados(n))
        pos = pos + 1
        n = int(datos[pos])
```

C++: feynman.cpp

Completo

```cpp
// suma de m * m para m de 1 a n: los cuadrados de lado n - m + 1
int cuadrados(int n) {
    return n * (n + 1) * (2 * n + 1) / 6;
}

int main() {
    int n;
    scanf("%d", &n);
    while (n != 0) {
        printf("%d\n", cuadrados(n));
        scanf("%d", &n);
    }
    return 0;
}
```

¿Cabe en int?

El producto más grande es $100 \cdot 101 \cdot 201 = 2\,030\,100$, lejos del límite de 32 bits.

Trampas

La esquina hasta $N - \ell$ excluido

Con `range(n - lado)` se pierde la última posición en cada lado. Con $N = 8$ el programa imprime 140, que es $\sum_{m=1}^{7} m^2$, en lugar de 204.

Contar solo los de lado 1

N^2 da 4 para $N = 2$, no 5: el enunciado pide todos los tamaños.

Procesar el 0

Imprimir una respuesta para el 0 agrega una línea que el juez no espera.

Variante para pensar

Rectángulos

¿Cuántos rectángulos hay en la cuadrícula de $N \times N$, contando los cuadrados? Con $N = 2$ son 9.

Pista

Congele el ancho a y el alto h: hay $(N - a + 1)(N - h + 1)$ posiciones para la esquina. La suma doble se separa en el producto de dos sumas iguales.

Contenido

1 Contar operaciones

2 Recursión y árbol de llamadas
- Concepto
- Ejemplo
- Ejercicio: UVa 839

3 Recursión por índices y su costo

4 Tarea del bloque

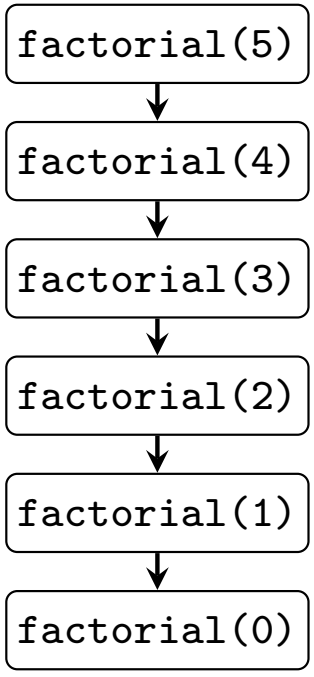
Lo que ya está: la pila de llamadas

De las estructuras y su costo

Cada llamada empuja un marco con sus parámetros y sus variables locales, y al terminar lo saca. La pila tiene un tamaño fijo, típicamente 8 MB en Linux, y Python corta a los 1000 niveles. En el depurador, `factorial(5)` detenido en el caso base:

```
(gdb) backtrace
#0  factorial (n=0) at factorial.cpp:8
#1  0x0000555555555168 in factorial (n=1) at factorial.cpp:10
#2  0x0000555555555168 in factorial (n=2) at factorial.cpp:10
#3  0x0000555555555168 in factorial (n=3) at factorial.cpp:10
#4  0x0000555555555168 in factorial (n=4) at factorial.cpp:10
#5  0x0000555555555168 in factorial (n=5) at factorial.cpp:10
#6  0x0000555555555193 in main () at factorial.cpp:17
```

Del `backtrace` al árbol de llamadas



El árbol

Cada llamada es un nodo y sus hijos son las llamadas que hace, de izquierda a derecha en el orden en que las hace. `factorial` hace una sola llamada, así que su árbol es una cadena de seis nodos.

La pila es un camino del árbol

En cada momento, la pila guarda el camino desde la raíz hasta la llamada que está corriendo: una llamada pendiente por nivel. El `backtrace` de arriba es ese camino leído de abajo hacia arriba.

Del `backtrace` al árbol de llamadas

Los dos casos

`n == 0` es el caso base: devuelve 1 sin llamar a nadie. El otro es el caso recursivo, `n * factorial(n - 1)`, que llama sobre una entrada más pequeña y así llega al base.

Lo que se lee en el árbol

- El número de nodos es el número de llamadas. Si cada llamada trabaja una cantidad constante fuera de sus llamadas, ese número es el costo.
- La altura del árbol, más uno, es el mayor número de marcos en la pila a la vez: lo que decide si la recursión cabe en 8 MB o en los 1000 niveles de Python.
- El orden en que se visitan los nodos, de arriba hacia abajo y de izquierda a derecha, es el orden en que se hacen las llamadas.

Dos llamadas por nivel

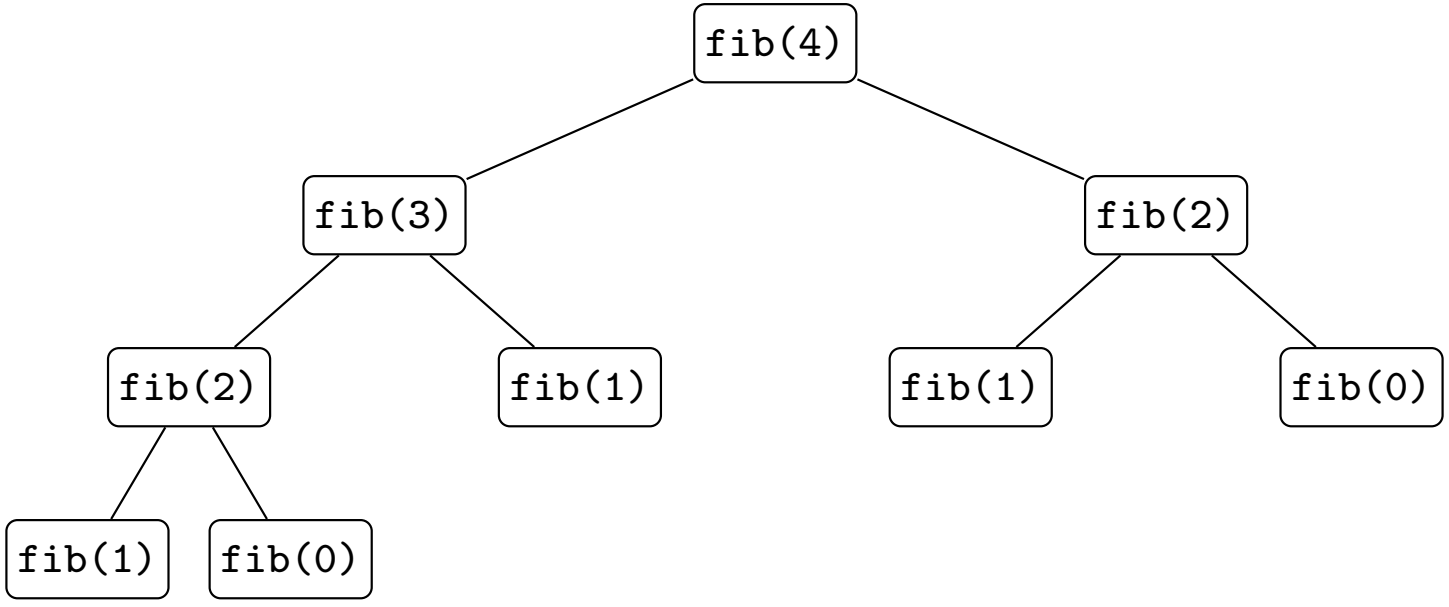
Python: `fib.py`

```python
def fib(n):
    if n <= 1:
        r = n
    else:
        r = fib(n - 1) + fib(n - 2)
    return r
```

Pregunta

Dibuje el árbol de llamadas de `fib(4)`. ¿Cuántas llamadas hay? ¿Cuántos marcos de `fib` hay en la pila como máximo?

El árbol de `fib(4)`



Respuesta

Nueve llamadas, y la función devuelve 3. El camino más largo baja por `fib(4)`, `fib(3)`, `fib(2)` y `fib(1)`: cuatro marcos a la vez. `fib(2)` se calcula dos veces, una en cada rama.

La pila mientras se recorre el árbol

Paso	Pila, del fondo al tope	Devuelve
1	`fib(4)`	
2	`fib(4)`, `fib(3)`	
3	`fib(4)`, `fib(3)`, `fib(2)`	
4	`fib(4)`, `fib(3)`, `fib(2)`, `fib(1)`	1
5	?	?
6	?	?
7	`fib(4)`, `fib(2)`	
8	?	?
9	?	?

Pregunta

En el paso 6, ¿qué llamada entra y qué tiene encima de ella?

La pila mientras se recorre el árbol

Paso	Pila, del fondo al tope	Devuelve
1	`fib(4)`	
2	`fib(4)`, `fib(3)`	
3	`fib(4)`, `fib(3)`, `fib(2)`	
4	`fib(4)`, `fib(3)`, `fib(2)`, `fib(1)`	1
5	`fib(4)`, `fib(3)`, `fib(2)`, `fib(0)`	0
6	`fib(4)`, `fib(3)`, `fib(1)`	1
7	`fib(4)`, `fib(2)`	
8	`fib(4)`, `fib(2)`, `fib(1)`	1
9	`fib(4)`, `fib(2)`, `fib(0)`	0

En el paso 6

`fib(2)` ya devolvió 1 y salió de la pila. Entra `fib(1)`, el segundo hijo de `fib(3)`, y no tiene nada encima: es un caso base.

UVa 839: Not so Mobile

Enunciado, en corto

Un móvil es una barra colgada de un hilo, con un objeto en cada extremo. Está en equilibrio si $W_l \times D_l = W_r \times D_r$, con los pesos W y las distancias D al punto de donde cuelga. En lugar de un objeto, de un extremo puede colgar otro móvil, y su peso es la suma de los pesos de sus objetos. Se pide decidir si todo el móvil está en equilibrio: cada una de sus barras.

Entrada

El número de casos, una línea en blanco, y entre caso y caso otra línea en blanco. Cada barra es una línea `Wl Dl Wr Dr`. Un peso 0 dice que de ese extremo cuelga un submóvil, descrito en las líneas que siguen: primero el izquierdo y después el derecho.

UVa 839: Not so Mobile

Salida

`YES` o `NO` por caso, con una línea en blanco entre dos casos.

Direcciones

Enunciado: `https://onlinejudge.org/external/8/839.pdf`

Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=780`

¿Entendimos el problema?

Muestra

```
1

0 2 0 4
0 3 0 1
1 1 1 1
2 4 4 2
1 6 3 2
```

Salida

```
YES
```

Pregunta

La primera barra tiene dos ceros. ¿Qué líneas describen el submóvil izquierdo y cuáles el derecho?

Pregunta

¿Cuánto pesa el submóvil izquierdo?

¿Entendimos el problema?

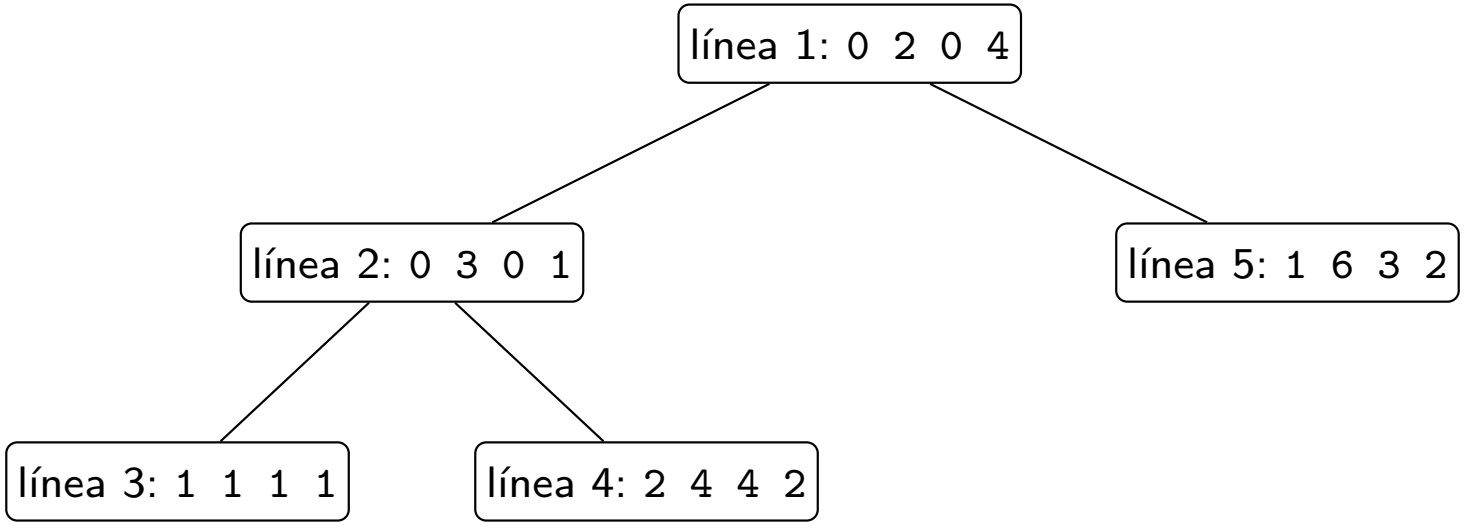
Respuesta

La línea 2, `0 3 0 1`, también tiene dos ceros: su submóvil izquierdo es la línea 3 y su derecho la línea 4. Con eso termina el submóvil izquierdo de la línea 1, y la línea 5 es el derecho. Las líneas no dicen dónde termina cada submóvil: hay que leerlas en orden y saber cuántas consume cada uno.

El peso

La línea 3 pesa $1 + 1 = 2$ y la 4, $2 + 4 = 6$; el submóvil izquierdo pesa 8. Su barra está en equilibrio porque $2 \cdot 3 = 6 \cdot 1$.

El móvil de la muestra



Las líneas y el árbol

Cada barra es un nodo, y cada peso 0 es un hijo. La entrada lista los nodos en el orden de arriba hacia abajo y de izquierda a derecha: la barra, después todo su lado izquierdo y después todo su lado derecho.

Primera idea: mirar cada barra sola

Pregunta

La línea 1 es `0 2 0 4`. ¿Se puede decidir si está en equilibrio con esos cuatro números?

Respuesta

No: $0 \cdot 2 = 0 \cdot 4$ siempre da verdadero, y los pesos de verdad están en las líneas de abajo. La barra necesita, de cada submóvil, dos cosas: cuánto pesa y si está en equilibrio por dentro.

La función

`movil` lee una barra. Por cada extremo con peso 0 se llama a sí misma, que lee todas las líneas de ese submóvil y devuelve su peso y si está en equilibrio. Con los dos pesos de verdad compara $W_l D_l$ con $W_r D_r$ y devuelve $W_l + W_r$ y el resultado.

Caso base y caso recursivo

Caso base

Un extremo con objeto: su peso viene en la línea y no hay nada que leer.

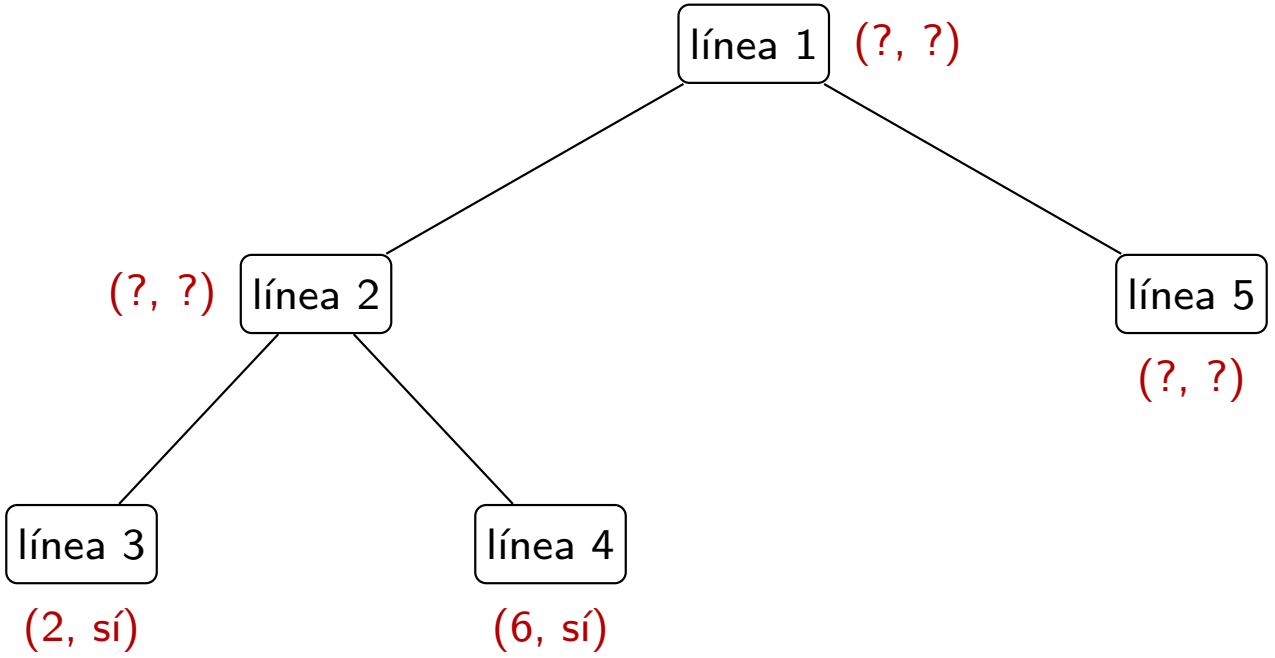
Caso recursivo

Un extremo con peso 0: una llamada que lee el submóvil completo. El submóvil tiene menos barras que el móvil, así que las llamadas terminan.

El árbol de llamadas es el móvil

Una llamada por barra, y cada llamada hace tantas llamadas como ceros tenga su línea: ninguna, una o dos.

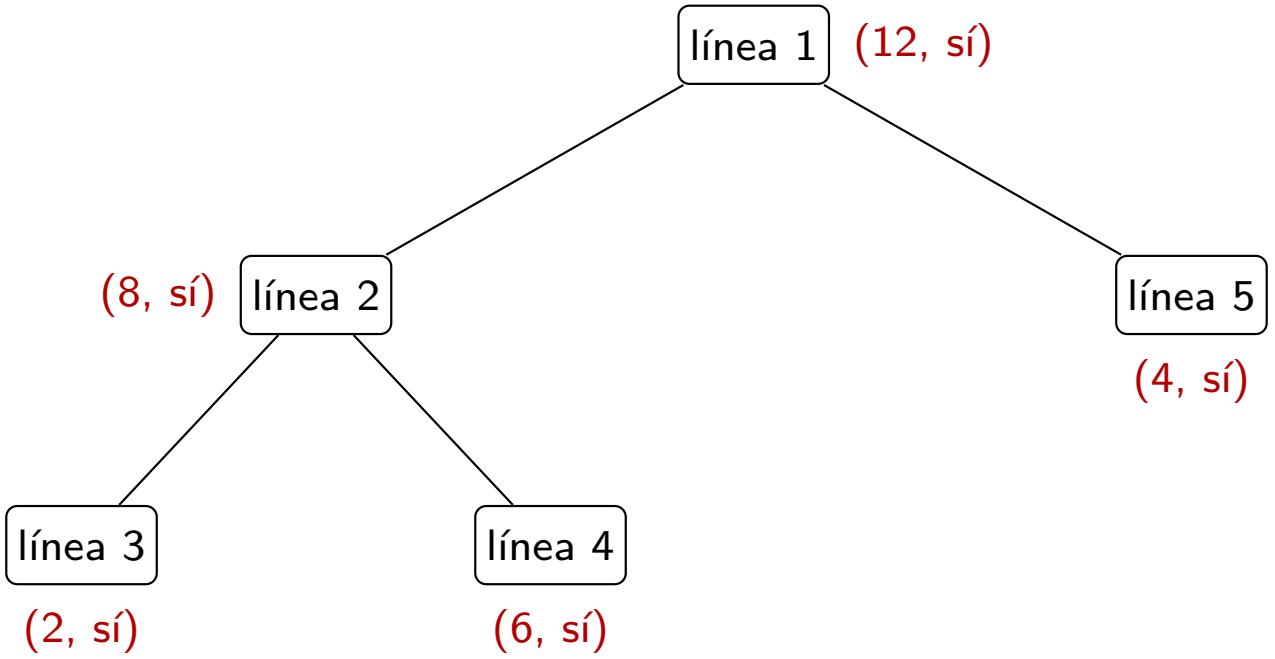
Lo que devuelve cada llamada



Pregunta

Las hojas ya devolvieron su par (peso, equilibrio). Complete los otros tres, de abajo hacia arriba.

Lo que devuelve cada llamada



Respuesta

Línea 2: $2 \cdot 3 = 6 \cdot 1$, pesa 8. Línea 5: $1 \cdot 6 = 3 \cdot 2$, pesa 4. Línea 1: con los pesos de verdad, $8 \cdot 2 = 4 \cdot 4$; todo el móvil pesa 12 y la salida es `YES`.

La pila mientras se lee la muestra

Paso	Pila, del fondo al tope	Lee	Devuelve
1	línea 1	`0 2 0 4`	
2	línea 1, línea 2	`0 3 0 1`	
3	línea 1, línea 2, línea 3	`1 1 1 1`	(2, sí)
4	?	?	?
5	línea 1, línea 2		?
6	?	?	?
7	línea 1		?

Pregunta

¿Cuál es el mayor número de marcos de `movil` en la pila? ¿Qué tiene que ver con el dibujo del móvil?

La pila mientras se lee la muestra

Paso	Pila, del fondo al tope	Lee	Devuelve
1	línea 1	0 2 0 4	
2	línea 1, línea 2	0 3 0 1	
3	línea 1, línea 2, línea 3	1 1 1 1	(2, sí)
4	línea 1, línea 2, línea 4	2 4 4 2	(6, sí)
5	línea 1, línea 2		(8, sí)
6	línea 1, línea 5	1 6 3 2	(4, sí)
7	línea 1		(12, sí)

Respuesta

Tres marcos, en los pasos 3 y 4: tantos como niveles tiene el móvil. Una línea se lee cuando su llamada entra a la pila, y una llamada sale solo cuando ya leyó todo lo que cuelga de ella.

Costo

Una llamada por línea y trabajo constante en cada una. Con n líneas y lados de a y b líneas, $a + b = n - 1$ y $T(n) = T(a) + T(b) + c$, con $T(0) = 0$: $T(n) = cn$, sea cual sea la forma del móvil. La pila llega a la altura del móvil.

Python: not-so-mobile.py

La función: devuelve el peso, el equilibrio y dónde sigue la entrada

```
def movil(datos, pos):
    # lee la barra que empieza en datos[pos] y todo lo que cuelga de ella;
    # devuelve el peso, si está en equilibrio y dónde sigue la entrada
    wl = int(datos[pos])
    dl = int(datos[pos + 1])
    wr = int(datos[pos + 2])
    dr = int(datos[pos + 3])
    pos = pos + 4
    equilibrio = True
    if wl == 0:
        wl, eq_izq, pos = movil(datos, pos)
        equilibrio = equilibrio and eq_izq
    if wr == 0:
        wr, eq_der, pos = movil(datos, pos)
        equilibrio = equilibrio and eq_der
    if wl * dl != wr * dr:
        equilibrio = False
    return wl + wr, equilibrio, pos
```

Python: not-so-mobile.py

Los casos

```
def main():
    sys.setrecursionlimit(10000)
    datos = sys.stdin.read().split()
    casos = int(datos[0])
    pos = 1
    for c in range(casos):
        peso, equilibrio, pos = movil(datos, pos)
        if c > 0:
            print("")
        if equilibrio:
            print("YES")
        else:
            print("NO")
```

sys.setrecursionlimit

Un móvil con más de mil niveles pasaría el límite de Python. El enunciado no da la altura máxima, y subir el límite a 10 000 cubre ese caso.

C++: not-so-mobile.cpp

La función: devuelve el peso y apaga equilibrio si una barra falla

```cpp
long long movil(bool& equilibrio) {
    long long wl, dl, wr, dr;
    scanf("%lld %lld %lld %lld", &wl, &dl, &wr, &dr);
    if (wl == 0) {
        wl = movil(equilibrio);
    }
    if (wr == 0) {
        wr = movil(equilibrio);
    }
    if (wl * dl != wr * dr) {
        equilibrio = false;
    }
    return wl + wr;
}
```

C++: not-so-mobile.cpp

Los casos

```
int main() {
    int casos;
    scanf("%d", &casos);
    for (int c = 0; c < casos; c++) {
        bool equilibrio = true;
        movil(equilibrio);
        if (c > 0) {
            printf("\n");
        }
        if (equilibrio) {
            printf("YES\n");
        } else {
            printf("NO\n");
        }
    }
    return 0;
}
```

Trampas

Dejar de leer cuando ya se sabe que es `NO`

Si la función no lee el submóvil derecho cuando el izquierdo ya falló, sus líneas quedan sin consumir y el caso siguiente arranca en mitad de este. Con los seis casos de `2.in`:

	1	2	3	4	5	6
esperado	NO	YES	NO	NO	NO	YES
sin leer el resto	NO	YES	YES	YES	NO	YES

Trampas

Las líneas en blanco

Se lee por palabras: los saltos de línea y las líneas vacías no cuentan.

La salida

La línea en blanco va entre dos casos, no después del último.

Pesos sin cota

El enunciado no acota los pesos ni las distancias, y la comparación multiplica un peso acumulado por una distancia: en C++ van en `long long`.

Revisar solo la barra de arriba

Un móvil cuya barra de arriba está en equilibrio y alguna de abajo no, se responde con `NO`: el caso 5 de `2.in` es así.

Variante para pensar

Cuántas barras fallan

En lugar de `YES` o `NO`, imprima cuántas barras del móvil no están en equilibrio. En el caso 5 de `2.in` son 2.

Pista

La función devuelve el peso y un contador en lugar del par. El contador de una barra es el de cada submóvil más 1 si ella misma falla.

Contenido

1 Contar operaciones

2 Recursión y árbol de llamadas

3 Recursión por índices y su costo
- Concepto
- Ejemplo
- Ejercicio: UVa 11173

4 Tarea del bloque

Un rango en vez de una copia

La función recibe `ini` y `fin`

La llamada trabaja sobre las posiciones $[ini, fin)$: desde *ini* incluida hasta *fin* excluida, $m = fin - ini$ posiciones. Partir el rango en dos es calcular $mitad = (ini + fin)/2$, una operación.

Copiar cuesta lo que se copia

`A[ini:mitad]` en Python, o un `vector` nuevo con esa mitad en C++, copia $m/2$ elementos. Ese costo se paga en cada llamada y cambia la recurrencia.

La recurrencia sale del código

Tres preguntas a la función

¿Cuántas llamadas hace? ¿Sobre qué tamaño? ¿Cuánto trabaja fuera de ellas? Con las respuestas se escribe $T(m)$; el caso base da $T(1) = c$.

Llamadas y trabajo fuera	$T(m)$	Crece como
una, sobre $m-1$; constante	$T(m-1)+c$	m
una, sobre $m/2$; constante	$T(m/2)+c$	$\log_2 m$
dos, sobre $m/2$; constante	$2T(m/2)+c$	m
dos, sobre $m/2$; copiar m	$2T(m/2)+cm$	$m \log_2 m$

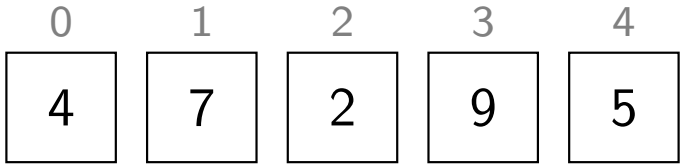
Cómo se resuelven

Por sustitución y con el árbol de recursión, y la solución se comprueba por inducción.

El máximo partiendo en mitades

Python: `maximo.py`

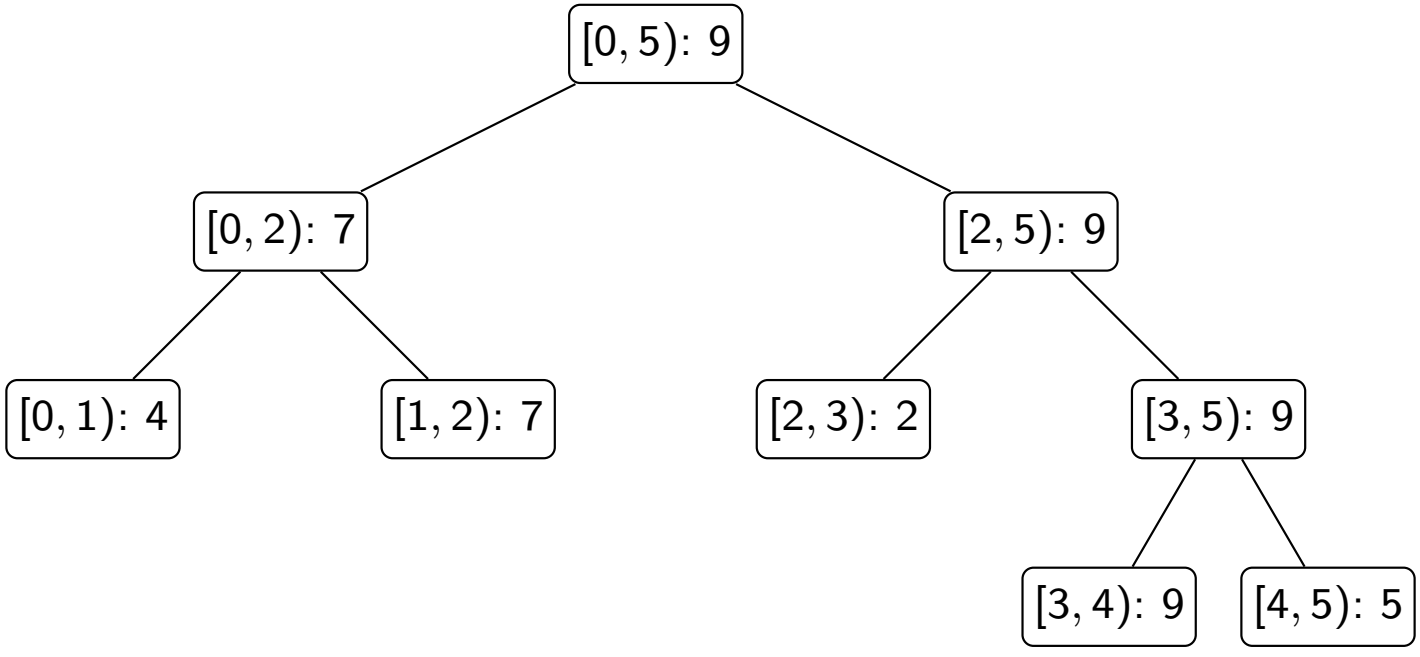
```python
def maximo(A, ini, fin):
    # mayor valor de A[ini..fin), con fin - ini >= 1
    if fin - ini == 1:
        r = A[ini]
    else:
        mitad = (ini + fin) // 2
        r = max(maximo(A, ini, mitad), maximo(A, mitad, fin))
    return r
```



Pregunta

Dibuje el árbol de llamadas de `maximo(A, 0, 5)`, con el rango de cada llamada. ¿Cuántas llamadas hay?

El árbol de `maximo(A, 0, 5)`



Respuesta

Nueve llamadas: cinco hojas, una por casilla, y cuatro nodos que juntan dos resultados. Con m casillas son $2m - 1$. Nadie copia el arreglo: cada nodo solo guarda dos números.

El árbol de `maximo(A, 0, 5)`

La recurrencia

Dos llamadas sobre la mitad y trabajo constante fuera: $T(m) = 2T(m/2) + c$. Como el árbol tiene $2m - 1$ nodos de costo constante, $T(m)$ crece como m.

Y la pila

El árbol tiene altura $\lceil \log_2 m \rceil$: con un millón de casillas, 21 marcos como máximo. La recursión cabe en Python sin tocar el límite.

La misma función, copiando

Python: `maximo.py`

```python
def maximo_copiando(A):
    if len(A) == 1:
        r = A[0]
    else:
        mitad = len(A) // 2
        r = max(maximo_copiando(A[:mitad]), maximo_copiando(A[mitad:]))
    return r
```

Pregunta

¿Qué cambia en la recurrencia?

La misma función, copiando

Respuesta

Las dos rebanadas copian m elementos entre las dos: $T(m) = 2T(m/2) + cm$. Cada nivel del árbol copia el arreglo completo, y hay $\log_2 m$ niveles: con $m = 2^{20}$ son $20 \cdot 2^{20} = 20\,971\,520$ elementos copiados.

Medido (Python 3.14, Ryzen 5 3600)

m	por índices	copiando
2^{20}	0,29 s	0,51 s
2 000 000	0,52 s	1,03 s
4 000 000	1,06 s	2,24 s

Lo que dice la tabla

Al duplicar m, la versión por índices duplica su tiempo; la que copia lo duplica y un poco más, por el factor $\log_2 m$. En Python la copia la hace código en C y por eso la diferencia se ve pequeña; en la búsqueda binaria, copiar convierte $\log_2 m$ en m.

UVa 11173: Grey Codes

Enunciado, en corto

El código de Gray reflejado de 1 bit es $0, 1$. El de n bits se arma con el de $n - 1$: arriba, en orden y con un 0 delante; abajo, reflejado y con un 1 delante. Se pide el número que queda en la posición k, contando desde 0.

Límites

Hasta 250 000 casos, cada uno con $1 \leq n \leq 30$ y $0 \leq k < 2^n$.

Direcciones

Enunciado: `https://onlinejudge.org/external/111/11173.pdf`

Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=2114`

¿Entendimos el problema?

k	bits	valor
0	000	0
1	001	1
2	011	3
3	010	2
4	110	6
5	111	7
6	101	5
7	100	4

La tabla del enunciado, $n = 3$

La muestra pide las ocho posiciones de $n = 3$, además de las de $n = 1$ y $n = 2$.

Pregunta

Mire las posiciones 4 a 7 sin el primer bit. ¿Qué lista aparece?

¿Entendimos el problema?

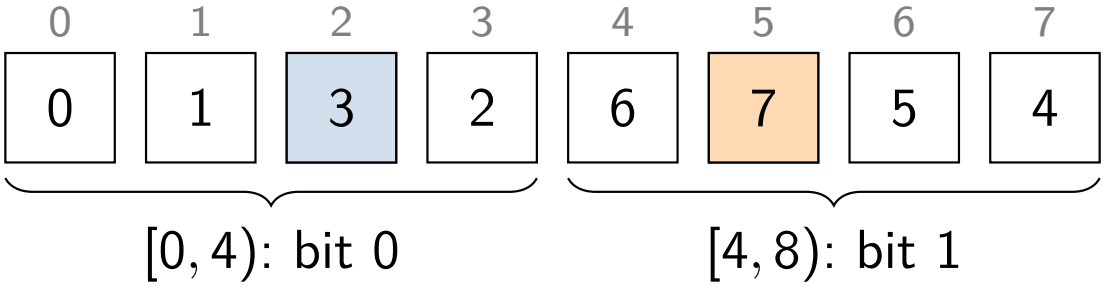
Respuesta

$10, 11, 01, 00$: el código de 2 bits, $00, 01, 11, 10$, al revés. La mitad de abajo es la de arriba reflejada, con un 1 delante, que vale 2^{n-1}.

Primera idea: construir la lista

Armar los 2^n códigos y leer la posición k. Con $n = 30$ son $1\,073\,741\,824$ números por caso, y puede haber $250\,000$ casos. No cabe ni en tiempo ni en memoria.

¿Dónde cae k?



Pregunta

$k = 5$ cae en la mitad de abajo. ¿Con qué posición de la mitad de arriba se corresponde? Las dos casillas marcadas dan una pista: $7 = 4 + 3$.

¿Dónde cae k?

Respuesta

Con la posición 2. El reflejo manda la primera de abajo a la última de arriba, y en general k a $ini + fin - 1 - k$: $0 + 8 - 1 - 5 = 2$. El código de la posición 5 es $4+$ el código de la posición 2 dentro de $[0, 4)$.

La función sobre el rango $[ini, fin)$

- Caso base: un rango de una posición tiene el código 0.
- $k < mitad$: bit 0, y se sigue con $[ini, mitad)$ y la misma k.
- $k \geq mitad$: bit 1, que vale $mitad - ini$, y se sigue con $[ini, mitad)$ y la posición reflejada.

Traza: $n = 3$, $k = 5$

$[ini, fin)$	k	*mitad*	¿mitad de abajo?	suma
$[0, 8)$	5	4	sí	4
$[0, 4)$	2	?	?	?
$[0, 2)$	?	?	?	?
$[0, 1)$	?		caso base	0

Traza: $n = 3$, $k = 5$

$[ini, fin)$	k	*mitad*	¿mitad de abajo?	suma
$[0, 8)$	5	4	sí	4
$[0, 4)$	2	2	sí	2
$[0, 2)$	1	1	sí	1
$[0, 1)$	0		caso base	0

Respuesta: $4 + 2 + 1 + 0 = 7$

Como en la tabla del enunciado. La posición reflejada de cada fila es la k de la siguiente: $0 + 4 - 1 - 2 = 1$ y $0 + 2 - 1 - 1 = 0$.

Costo

Una llamada sobre la mitad y trabajo constante: $T(m) = T(m/2) + c$, con $m = 2^n$. El rango pasa por $2^n, 2^{n-1}, \ldots, 1$: $n + 1$ llamadas por caso, 31 como máximo.

Python: grey-codes.py

La función

```
def gris(ini, fin, k):
    # código que ocupa la posición k del rango [ini, fin) de posiciones
    if fin - ini == 1:
        codigo = 0
    else:
        mitad = (ini + fin) // 2
        if k < mitad:
            # mitad de arriba: bit 0, mismo orden
            codigo = gris(ini, mitad, k)
        else:
            # mitad de abajo: bit 1, la de arriba reflejada
            codigo = (mitad - ini) + gris(ini, mitad, ini + fin - 1 - k)
    return codigo
```

Python: `grey-codes.py`

Los casos: una sola escritura al final

```
def main():
    datos = sys.stdin.read().split()
    casos = int(datos[0])
    salida = []
    for c in range(casos):
        n = int(datos[1 + 2 * c])
        k = int(datos[2 + 2 * c])
        salida.append(str(gris(0, 2 ** n, k)))
    print("\n".join(salida))
```

Medido (Python 3.14, Ryzen 5 3600)

250 000 casos con n al azar entre 1 y 30: 0,94 s en Python y 0,11 s en C++.

C++: grey-codes.cpp

La función

```cpp
long long gris(long long ini, long long fin, long long k) {
    long long codigo;
    if (fin - ini == 1) {
        codigo = 0;
    } else {
        long long mitad = (ini + fin) / 2;
        if (k < mitad) {
            // mitad de arriba: bit 0, mismo orden
            codigo = gris(ini, mitad, k);
        } else {
            // mitad de abajo: bit 1, la de arriba reflejada
            codigo = (mitad - ini) + gris(ini, mitad, ini + fin - 1 - k);
        }
    }
    return codigo;
}
```

C++: grey-codes.cpp

Los casos: 2^n con un ciclo

```
int main() {
    int casos;
    scanf("%d", &casos);
    for (int c = 0; c < casos; c++) {
        int n;
        long long k;
        scanf("%d %lld", &n, &k);
        long long fin = 1;
        for (int i = 0; i < n; i++) {
            fin = fin * 2;
        }
        printf("%lld\n", gris(0, fin, k));
    }
    return 0;
}
```

Trampas

Restar la mitad en lugar de reflejar

Con $k - (\mathit{mitad} - \mathit{ini})$ en la mitad de abajo, la función devuelve el mismo k: para $n = 3$ imprime $0, 1, 2, \ldots, 7$, el número en binario. La mitad de abajo va al revés.

Las posiciones empiezan en 0

La primera posición es $k = 0$, como en la tabla del enunciado.

Mucha salida

Con 250 000 líneas, Python arma la salida en una lista y la imprime una vez.

Variante para pensar

Al revés

Dado n y un código g, ¿en qué posición del código de Gray de n bits está? Con $n = 3$ y $g = 7$, la respuesta es 5.

Pista

El código está en la mitad de abajo si y solo si $g \geq \mathit{mitad} - \mathit{ini}$. La recursión tiene la misma forma y la misma recurrencia.

Contenido

1 Contar operaciones

2 Recursión y árbol de llamadas

3 Recursión por índices y su costo

4 Tarea del bloque

Para el bloque 3

Cómo se trabajan

Los tres se resuelven, se envían al juez y se revisan al comienzo del bloque siguiente. Van en orden de dificultad.

Python en UVa

El juez corre una versión antigua de Python 3: la salida se arma con `print` y con `%`, como en las soluciones de hoy.

Base. UVa 10079: Pizza Cutting

Direcciones

Enunciado: `https://onlinejudge.org/external/100/10079.pdf`

Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=1020`

Cómo atacarlo

El corte k cruza a los $k-1$ anteriores y pasa por k pedazos, así que agrega k: el total es $1+\sum_{k=1}^{N} k = 1 + N(N+1)/2$. Con N hasta $210\,000\,000$, el ciclo que suma no alcanza y la fórmula responde en $O(1)$.

La trampa

El resultado pasa de 32 bits. Con $N = 0$ hay un pedazo, y la entrada termina con cualquier número negativo, no solo con -1.

Intermedio. UVa 913: Joana and the Odd Numbers

Direcciones

Enunciado: `https://onlinejudge.org/external/9/913.pdf`

Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=854`

Cómo atacarlo

La fila con N números es la $L = (N+1)/2$, y hasta el final de esa fila hay $\sum_{i=1}^{L}(2i-1) = L^2$ impares escritos. El último es $2L^2 - 1$ y los tres últimos suman $6L^2 - 9$: para $N = 3$ son 15, como en la muestra. Recorrer las filas no alcanza, porque N llega a 10^9.

La trampa

$6L^2$ se pasa de 32 bits; el enunciado garantiza que cabe en 64.

Exigente. UVa 620: Cellular Structure

Direcciones

Enunciado: `https://onlinejudge.org/external/6/620.pdf`

Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=561`

Cómo atacarlo

Una cadena sana es `A`, una sana seguida de `AB`, o `B` seguida de una sana y de `A`. La función recibe el rango $[ini, fin)$: el caso base es una letra, y la última letra decide qué caso recursivo se intenta, `B` el primero y `A` el segundo. Una llamada por nivel, $T(n) = T(n-2) + c$; cortar la cadena en cada llamada lo deja en $\Theta(n^2)$.

La trampa

Se imprime la etapa de la regla de afuera; lo de adentro solo tiene que estar sano. `BAABA` es `MUTAGENIC` porque `AAB` es sana, y `BAAB` es `MUTANT` porque `BA` no es sana.

Para cerrar el bloque

Búsqueda binaria por índices

Escriba en Python y en C++ la búsqueda binaria recursiva sobre $[ini, fin)$ de un arreglo ordenado, sin copiar. Plantee su $T(m)$ a partir del código y cuente sus llamadas con $m = 16$ cuando el valor no está.